Screen-Space Perceptual Rendering of Human Skin

Real-Time Rendering - Assignment 5

Diksha Chottani

April 2026

Contents

Abstract

This report describes the implementation of the screen-space perceptual skin rendering technique proposed by Jimenez, Sundstedt, and Gutierrez [Jimenez et al.(2009)] using C++ and OpenGL 3.3. The technique approximates subsurface scattering in human skin as a screen-space post-processing effect, avoiding the per-character texture-space operations required by physically correct methods. The implemented pipeline comprises four passes: a geometry pass writing diffuse, specular, and depth information to a G-buffer; a horizontal depth-modulated separable Gaussian blur; a vertical depth-modulated separable Gaussian blur; and a composite pass combining the blurred diffuse with the unblurred specular channel. The implementation was developed and tested on an Apple M2 system using a UV sphere as the test geometry. The report documents the pipeline architecture, shader design, hardware constraints encountered during development, and an honest evaluation of the visual result. Limitations relative to the paper's reference implementation are identified, and theoretical improvements are proposed.

1 Background

1.1 The Problem with Rendering Skin

Human skin presents one of the most challenging materials to render convincingly in real time. Unlike opaque surfaces such as metal or painted wood, skin is a translucent, multi-layered biological material. When a light strikes the surface, a significant portion of it is not immediately reflected. Instead, it penetrates the outer epidermis, scatters through the underlying dermis, and exits at a point spatially displaced from its entry point. This phenomenon is known as subsurface scattering (SSS).

The visual consequence of subsurface scattering is a characteristic softness in skin's appearance under direct illumination; shadows are not hard-edged, the lit-to-shadow transition is gradual and warm, and thin regions such as ears and fingertips exhibit visible light transmission. Standard real-time shading models, most notably Lambertian diffuse combined with Blinn-Phong specular, treat each surface point independently. They compute the outgoing radiance at a pixel, with no account for light that has scattered from neighboring regions beneath the surface. The result is a characteristically flat, plastic appearance that viewers immediately perceive as artificial.

1.2 Prior Work and its Limitations

The physically accurate treatment of subsurface scattering in skin was formalized by Jensen et al. [Jensen et al.(2001)], who introduced the dipole diffusion approximation, a physically grounded model treating the dermis as a participating medium with defined scattering and absorption coefficients. While accurate, this model was prohibitively expensive for real-time applications.

The closest prior real-time solution was texture-space diffusion, developed by d'Eon et al. [d'Eon et al.(2007)]. In this approach, the scene's irradiance is first rendered into a texture atlas in UV space. That texture is then blurred using a sum of Gaussian kernels fit to the dipole diffusion profile and the result is re-projected back onto the mesh during final shading. While this method produces perceptually convincing skin, it carries two significant practical limitations. First, the computational cost scales linearly with the number of characters visible on screen, since each character requires its own irradiance texture and blur pass. Second, the technique depends on a well-parameterized, low-distortion UV unwrap for every mesh, making it difficult to apply generically across a scene with varied geometry.

1.3 The Paper's Contribution

Jimenez et al. [Jimenez et al.(2009)] propose an alternative that moves the scattering computation entirely into screen space. Rather than blurring irradiance in UV space per character, a depth-modulated separable Gaussian blur is applied directly to the diffuse channel of the rendered image as a post-processing step. The specular component is separated in the G-buffer and remains unblurred throughout the pipeline, preserving the sharpness of surface highlights. The blurred diffuse and unblurred specular are then composited to produce the final image.

Because the blur operates on the screen-space image rather than per-character textures, the cost of the technique is fixed with respect to scene population, rendering one character or ten incurs the same post-processing overhead. The technique requires no special UV parameterization and is straightforwardly applicable to arbitrary geometry.

The central academic contribution of the paper, however, is not technical but perceptual. The authors conducted a psychophysical user study in which participants were asked to distinguish between renders produced by the screen-space approximation and those produced by the physically correct texture-space method of d'Eon et al., under varying lighting conditions and viewing distances. Participants were unable to reliably differentiate the two methods. This finding validates the use of perceptual plausibility as a sufficient criterion for real-time skin rendering, establishing that physical correctness is not a prerequisite for visual believability.

2 Implementation Details

2.1 Technology Stack

The implementation was developed in C++ using OpenGL (core profile 3.3) for the GPU rendering pipeline. Scene management, the main render loop, camera and light controls, and all framebuffer

object (FBO) configuration were handled on the CPU side in C++. All shading logic including the geometry passes, and compositing was written in GLSL as fragment and vertex shaders.

The project was built using CMake. The following third-party libraries were used:

1. GLFW [GLFW Contributors(2024)] for window creation and input handling.
2. GLAD [Dav1dde(2024)] for OpenGL function loading
3. GLM [Pergo(2024)] for mathematics including vector and matrix operations.

2.2 Scene and Test Geometry

The paper by Jimenez et al. demonstrates their technique on a full human head mesh with photographic skin textures. For this implementation, a UV sphere with a uniform skin-tone color was used as the test geometry in place of a head mesh. This decision was made for three reasons.

First, Isolation: A UV sphere has a perfectly predictable surface and clean surface and clean geometry, which ensures that any visual artifacts observed during development are attributable to the rendering pipeline rather than to mesh or texture issues. This significantly simplified debugging across the multi-pass pipeline.

Second, Curvature: A sphere provides continuous, smooth curvature in all directions, producing a clearly visible terminator, the gradual transition from the lit region to the shadowed region. This boundary is precisely where subsurface scattering has the most pronounced visual effect, making the sphere an effective test case for validating the technique.

Third, Scope: Correctly loading, UV-unwrapping and texturing a high-polygon head mesh would have introduced a substantial amount of asset complexity that was outside the scope of validating the algorithm itself. The goal of this implementation was to demonstrate the screen-space SSS pipeline functioning correctly and the sphere was sufficient for that purpose.

2.3 The Render Pipeline

The pipeline consists of four sequential passes. The first pass renders the scene geometry into a set of textures known as the G-buffer. The subsequent two passes perform a separable Gaussian blur on the diffuse channel. The final pass composites the blurred diffuse with the unblurred specular to produce the output image. After the geometry pass, all remaining passes operate on fullscreen quads hence no geometry is revisited.

2.3.1 Pass 1 — Geometry Pass (G-buffer Write)

The sphere is rendered once into a framebuffer object with three attachments: a diffuse irradiance texture bound to `GL_COLOR_ATTACHMENT0`, specular texture bound to `GL_COLOR_ATTACHMENT1`, and a depth texture bound to `GL_DEPTH_ATTACHMENT`. The geometry fragment shader computes a Lambertian diffuse term and a Blinn-Phong specular tern and write them to their respective attachments simultaneously. The depth buffer is written automatically by OpenGL. After this pass, all three textures are available as inputs to subsequent passes.

2.3.2 Pass 2 — Horizontal Blur

A full screen quad is rendered to a new framebuffer. The fragment shader reads from the diffuse texture and the depth texture produced in Pass 1. For each fragment, a Gaussian kernel is evaluated by sampling neighboring pixels along the horizontal axis only. The contribution of each neighbor is weighted by both its Gaussian weight and a depth-edge guard, if the absolute difference between the neighbor's depth value and the center pixel's depth value exceeds a threshold, the weight is set to zero. This prevents the blur from spreading color across depth discontinuities such as silhouette edges, which would otherwise cause the skin color to bleed onto the background. The result is written to an intermediate texture.

2.3.3 Pass 3 — Vertical Blur

The vertical blur pass is structurally identical to Pass 2, with the sole difference that the kernel is sampled along the vertical axis. It reads from the output of the horizontal blur pass and the original depth texture, applying the same depth-edge guard. Performing the blur as two separate one-dimensional passes (horizontal then vertical) is mathematically equivalent to a single two-dimensional Gaussian blur, but reduces the number of texture samples from N^2 to $2N$ for a kernel of radius N, providing a substantial performance advantage.

2.3.4 Pass 4 — Composite

The final pass reads from the vertically blurred diffuse texture and the unblurred specular texture from the G-buffer, and adds them to produce the output color. A boolean uniform `uSSS` controls whether the blurred or the original diffuse texture is used, enabling the real-time toggling between standard diffuse shading and the full SSS approximation without any change to the specular contribution. This toggle was used during development to verify the effect of the blur and serves as the primary interactive demonstration of the technique.

2.4 Depth-Modulated Kernel Width

The blur kernel width is not fixed globally. As expressed in Equation 1, the kernel width w is proportional to the SSS strength parameter S_{strength} and inversely proportional to the linearized depth value d at the current fragment. A small constant ε is added to the denominator to prevent division by zero at near-zero depth values.

The practical effect of this modulation is that the apparent scatter radius scales with distance from the camera in a manner consistent with the physical intuition that subsurface scattering is a world-space phenomenon. Surfaces closer to the camera, which occupy more screen pixels per unit of world-space area, receive a proportionally wider screen-space blur to compensate.

$$w = \frac{S_{\text{strength}}}{d + \varepsilon} \tag{1}$$

3 Results and Evaluation

3.1 Visual Results

The effect of the screen-space SSS pipeline is most apparent at the terminator of the sphere, the gradual transition from the directly lit region to shadow. With SSS enabled, this transition exhibits a subtle softening and warm color bleed consistent with the appearance of real skin under direct illumination. However, the degree of visible difference between the SSS-enabled and SSS-disabled modes is modest on the test geometry used. This is attributable to two factors.

First, the test mesh is a uniformly colored UV sphere rather than a photographic skin texture. The paper's reference renders use a high-detail human head mesh with albedo and specular maps that provide strong local contrast, making the scattering effect considerably more pronounced. On a uniform base color, the blurred and unblurred diffuse channels are perceptually similar.

Second, the depth-modulated kernel width at the default SSS strength of $S_{\text{strength}} = 0.02$ produces a conservative scatter radius. Increasing this value toward the upper bound of 0.05 makes the effect noticeably more visible, though at the cost of some edge softening near the silhouette boundary.

The specular separation functions correctly in both modes, highlights remain sharp regardless of the SSS strength setting, confirming that the diffuse-specular split in the G-buffer is working as intended. The depth-edge guard similarly performs as expected, with no color bleeding from the sphere onto the background observed at any strength setting.

3.2 Performance

Performance was evaluated qualitatively on an Apple M2 (8-core GPU) at a resolution of 1100×800 pixels. The blur kernel uses a 5-tap separable Gaussian with weights $[0.2270, 0.1946, 0.1216, 0.0541, 0.0162]$, applied across 4 neighbors on each side of the center pixel in both the horizontal and vertical passes. The kernel width is computed per fragment as $w = S_{\text{strength}}/(d + 0.001)$, where d is the linearized depth at the center pixel and $S_{\text{strength}} = 0.02$ is the default SSS strength.

The Apple M2's integrated GPU presents notable constraints for this pipeline. Memory bandwidth is shared between the CPU and GPU, and OpenGL on Apple Silicon runs through a translation layer to the Metal backend, introducing overhead that would not be present on a discrete GPU with a native OpenGL driver. The multi-pass nature of the pipeline which requires four sequential fullscreen passes, two of which perform dense texture reads across `blur_h.frag` and `blur_v.frag` is particularly sensitive to memory bandwidth limitations.

Despite these constraints, the implementation runs interactively. The additional cost of the two blur passes and the composite pass is perceptible when compared to rendering with SSS disabled, but remains within an acceptable range for a real-time demonstration. This is consistent with the paper's claim that the screen-space approach carries a fixed post-processing overhead independent of scene complexity, a property that distinguishes it from texture-space methods whose cost scales with the number of characters in the scene.

4 Limitations and Improvements

4.1 Limitations of the Current Implementation

Several limitations are present in this implementation relative to the full technique described by Jimenez et al.

The most significant visual limitation is the use of a uniform, procedurally colored UV sphere as the test geometry in place of a human head mesh with photographic skin textures. The paper's perceptual evaluation and its reference renders both rely on a high-detail head mesh with spatially varying albedo, specular, and normal maps. These textures provide the local contrast necessary for subsurface scattering to produce a clearly distinguishable visual result. On a uniformly colored surface, the difference between a blurred and an unblurred diffuse channel is inherently subtle, and the full perceptual benefit of the technique is not realized.

The implementation does not support multiple simultaneous characters or meshes. While the screen-space approach is theoretically cost-invariant with respect to scene population, one of its primary advantages over texture-space diffusion. This property could not be demonstrated in practice due to the single object scene setup.

The Gaussian kernel used in both blur passes is a fixed 5-tap filter. Jimenez et al. fit their blur profile to the six-Gaussian sum defined by d'Eon et al. [d'Eon et al.(2007)], which more accurately approximates the spectral diffusion profile of real skin across different wavelengths. The simplified single-Gaussian kernel used here does not replicate the wavelength dependent scattering behavior described in the paper, where red light scatters significantly further beneath the skin surface than green or blue light.

Finally, the depth edge threshold in `blur_h.frag` uses an inconsistent value of 0.5 for the positive neighbor and 0.05 for the negative neighbor. This asymmetry was not intentional and introduces a directional bias in the horizontal blur at depth discontinuities. The positive direction samples are significantly more permissive than the negative direction samples when deciding whether to include a neighbor in the weighted sum.

4.2 Potential Improvements

The following improvements are proposed based on an understanding of the technique and the limitations identified above. None of these have been implemented or tested, and the observations below are theoretical rather than empirical.

The most impactful proposed change would be replacing the UV sphere with a human head mesh and accompanying photographic skin textures. Based on the reference renders in Jimenez et al., the spatial variation in a real albedo texture. Particularly, around features such as the ears, nostrils, and cheek boundary, it is likely to make the scattering effect considerably more perceptible than it is on a uniform surface color.

The single-Gaussian kernel could in principle be replaced with a weighted sum of Gaussians fit to the dipole diffusion profile, as used by Jimenez et al. This would introduce wavelength dependent scattering across the RGB channels, which the paper identifies as a key contributor to the characteristic warm appearance of skin under direct illumination. Whether this change would produce a perceptible difference on the current test geometry is unclear without testing.

The asymmetric depth threshold in `blur_h.frag` where the positive neighbor uses a threshold of 0.5 and the negative neighbor uses 0.05 should be corrected to use a consistent value in both directions, matching the behavior of `blur_v.frag`. This is a straightforward fix that would remove an unintended directional bias at depth discontinuities.

Finally, the translucency pass described in the paper, which approximates light transmitted through thin geometry, was not implemented. Including it would complete the pipeline as described by Jimenez et al., though its visual contribution on a solid sphere would likely be negligible without appropriate test geometry such as a hand or ear mesh.

5 Conclusion

This report has described the implementation of the screen-space perceptual skin rendering technique proposed by Jimenez et al. [Jimenez et al.(2009)] using C++ and OpenGL. The four-pass pipeline, comprising a geometry pass writing to a G-buffer, a horizontal depth-modulated Gaussian blur, a vertical depth-modulated Gaussian blur, and a composite pass combining the blurred diffuse with the unblurred specular, was successfully implemented and runs interactively on an Apple M2 system.

The central insight of the paper is not purely technical. By demonstrating through a psychophysical user study that observers cannot reliably distinguish the screen-space approximation from the physically correct texture-space method of d'Eon et al., Jimenez et al. establish that perceptual plausibility is a sufficient criterion for real-time skin rendering. This shifts the problem from one of physical simulation to one of perceptual engineering, and the screen-space formulation is a direct consequence of that shift. It is designed to be believable rather than accurate.

The implementation confirmed several of the technique's theoretical properties in practice. The post-processing cost is fixed with respect to scene complexity, requiring no per-character textures or UV parameterization. The depth-edge guard in the blur passes successfully prevents color bleeding across silhouette boundaries. The separation of the specular component in the G-buffer correctly preserves highlight sharpness independent of the blur strength.

The primary shortcoming of this implementation is the absence of photographic skin textures and a human head mesh, which are necessary for the scattering effect to be clearly perceptible. On a uniformly colored sphere the visual difference between SSS-enabled and SSS-disabled modes is subtle. Nevertheless, the pipeline architecture is complete and correct, and the effect becomes visible at higher SSS strength values and at the terminator boundary under direct illumination.

This implementation has provided a practical understanding of multi-pass deferred rendering pipelines, separable Gaussian blur as a post-processing technique, depth-based edge preservation in screen-space filters, and the broader principle that perceptual equivalence rather than physical correctness is often the appropriate target in real-time graphics.

References

[Dav1dde(2024)] Dav1dde. 2024. GLAD — Multi-Language GL/GLES/EGL/GLX/WGL Loader-Generator. https://glad.dav1d.de.

[d'Eon et al.(2007)] Eugene d'Eon, David Luebke, and Eric Enderton. 2007. Efficient Rendering of Human Skin. In *Proceedings of the Eurographics Symposium on Rendering*. Eurographics Association, 147–157. doi:10.2312/EGWR/EGSR07/147-157

[GLFW Contributors(2024)] GLFW Contributors. 2024. GLFW — A multi-platform library for OpenGL. https://www.glfw.org.

[Jensen et al.(2001)] Henrik Wann Jensen, Stephen R. Marschner, Marc Levoy, and Pat Hanrahan. 2001. A Practical Model for Subsurface Light Transport. *Proceedings of SIGGRAPH 2001* (2001), 511–518. doi:10.1145/383259.383319

[Jimenez et al.(2009)] Jorge Jimenez, Veronica Sundstedt, and Diego Gutierrez. 2009. Screen-Space Perceptual Rendering of Human Skin. *ACM Transactions on Applied Perception* 6, 4 (2009), 1–15. doi:10.1145/1609967.1609970

[Pergo(2024)] Christophe Pergo. 2024. GLM — OpenGL Mathematics. https://glm.g-truc.net.